

任务二项目测试报告

使用 TINY 语言的文法进行测试，对应的输入内容如下所示：

```
program | stmt-sequence | statement | if-stmt | repeat-stmt |  
assign-stmt | read-stmt | write-stmt | exp | simple-exp |  
comparison-op | addop | term | mulop | factor  
  
SEMI | if | then | end | else | repeat | until | ASSIGN | read |  
ID | number | write | LT | EQ | LTEQ | NE | RTEQ | RT | PLUS |  
MINUS | MULTIPLY | DIVIDE | MOD | LPAN | RPAN | NUMBER  
  
program -> stmt-sequence  
  
stmt-sequence -> stmt-sequence SEMI statement  
  
stmt-sequence -> statement  
  
statement -> if-stmt  
  
statement -> repeat-stmt  
  
statement -> assign-stmt  
  
statement -> read-stmt  
  
statement -> write-stmt  
  
if-stmt -> if exp then stmt-sequence end  
  
if-stmt -> if exp then stmt-sequence else stmt-sequence end  
  
repeat-stmt -> repeat stmt-sequence until exp  
  
assign-stmt -> ID ASSIGN exp  
  
read-stmt -> read ID  
  
write-stmt -> write exp  
  
exp -> simple-exp comparison-op simple-exp  
  
exp -> simple-exp  
  
comparison-op -> LT  
  
comparison-op -> EQ
```

```
comparison-op -> LTEQ
comparison-op -> NE
comparison-op -> RTEQ
comparison-op -> RT
simple-exp -> simple-exp addop term
simple-exp -> term
addop -> PLUS
addop -> MINUS
term -> term mulop factor
term -> factor
mulop -> MULTIPLY
mulop -> DIVIDE
mulop -> MOD
factor -> LPAN exp RPAN
factor -> NUMBER
factor -> ID
```

用于测试的 TINY 语言的源代码对应的 lex 文件内容如下所示:

测试 1:

```
106 400 "n" 211 400 "sum" 212 500 "0" 211 400 "i" 212 500 "1" 211
107 400 "sum" 211 100 400 "n" 204 500 "2" 210 500 "0" 101 107 500
"0" 102 107 500 "1" 103 211 104 400 "n" 212 400 "n" 201 500 "1"
211 107 400 "n" 105 400 "n" 210 500 "0" -1
```

测试 2:

```
400 "azusa" 212 400 "a" 204 500 "123" 200 500 "1" -1
```

基于数据准备中所提到的 TINY 文法和 TINY 测试程序源代码，把此二者输入后，依次点击按钮控件进行 FIRST 集合求解、FOLLOW 集合求解、LR(0)DFA 构建、SLR(1)文法判断、LR(1)DFA 生成、LR(1)分析表构建、测试源码的句子分析表构建、语法树可视化以及中间代码生成。



图 1 系统启动

在启动项目系统后，在“文法载入编辑”框内输入/载入上面的 tiny 文法规则，点击图形化窗口界面中的“First 集合求解”旁的“求解”按钮，即可进行 FIRST 集的构建并展示在图形化窗口界面对应区域中，供用户查看，具体如下图所示：

	非终结符	First集合
1	addop	MINUS,PLUS
2	assign-stmt	ID
3	comparison-op	EQ,LT,LTEQ,N...
4	exp	ID,PAR,NL,IM

图 2 FIRST 集构建测试（TINY）

由于显示区域大小问题，构建的 FIRST 集无法显示完全，但是可以通过滑动条实现完整的查看。

接下来，点击图形化窗口界面中的“Follow 集合求解”旁的“求解”按钮，即可进行 FOLLOW 集的构建并展示在图形化窗口界面对应区域中，供用户查看，具体如下图所示：

Follow集合求解

求解

	非终结符	Follow集合
1	addop	ID,LPAN,NUM...
2	assign-stmt	\$,SEMI,else,en...
3	comparison-op	ID,LPAN,NUM...
4	exp	\$(LPANSEMI

图 3 FOLLOW 集构建测试 (TINY)

同理，由于显示区域大小问题，构建的 FOLLOW 集无法显示完全，但是可以通过滑动条实现完整的查看。

接下来，点击“LR(0)DFA 生成”按钮，即可在下方的“分析结果表”内生成 LR(0)DFA 图（表格形式），同时可以查看上方的“对应文法”，来核对用户输入的文法是否一致，具体如下图所示：

SLR(1) 文法分析

源代码分析

分析结果表

	状态	状态内文法	ASSIGN	VII	EQ	ID	LP
1	0	program->.stmt-...				1	
2	1	assign-stmt-...	13				
3	2	statement->assign-stmt.					
4	3	if-stmt->if.expthenstmt-...				14	15
5	4	statement->if-stmt.					
6	5	read-stmt->read.ID				43	
7	6	statement->read-stmt.					
8	7	repeat-stmt-...				1	
9	8	statement->repeat-stmt.					
10	9	stmt-sequence-...					
11	10	program->stmt-...					

图 4 LR(0)DFA 图构建测试 (TINY)

由于显示区域大小问题，构建的 LR(0)DFA 图（表格形式）无法显示完全，但是可以通过滑动条实现完整的查看。

输入 TINY 文法，点击项目界面上方的“SLR(1)文法分析”按钮，即可开始 SLR(1)文法分析。

本次测试中使用的 TINY 语言的文法是 SLR(1)文法，故能通过 SLR(1)文法判断，同时我也通过局部修改 TINY 语言的文法来进行具有冲突情况的测试，具体操作如下：

- (1) 新增了非终结符 `primary`，并让 `term -> primary` 与已有的 `term -> factor` 并存，而 `primary` 与 `factor` 都能直接归约为 `NUMBER` 或 `ID`，由此构造了具有“规约-规约冲突”的文法；
- (2) 引入了经典的“悬挂 `else`”二义性，构造具有“移进-规约冲突”的文法；
- (3) 把 (1) 和 (2) 的内容相结合，构造出同时存在“移进-规约冲突”和“规约-规约冲突”的文法。

将这四个文法进行测试，具体测试结果如下：

文法分析

查看分析说明

LR(0)DFA生成

SLR(1)文法分析

LR(1)DFA生成

LR(1)分析表生成

源代码分析

该文法不是SLR(1)文法
原因：存在规约-规约冲突
在部分DFA状态中，存在多个可用的规约项目，且它们的Follow集合有交集

查看分析说明

LR(0)DFA生成

SLR(1)文法分析

文法分析

LR(1)DFA生成

LR(1)分析表生成

源代码分析

该文法不是SLR(1)文法
原因：存在移进-规约冲突
在部分DFA状态中，同一个终极符既可以移进也可以规约

查看分析说明

LR(0)DFA生成

SLR(1)文法分析

文法分析

LR(1)DFA生成

LR(1)分析表生成

源代码分析

该文法不是SLR(1)文法
原因：同时存在移进-规约冲突和规约-规约冲突

查看分析说明

LR(0)DFA生成

SLR(1)文法分析

文法分析

LR(1)DFA生成

LR(1)分析表生成

源代码分析

该文法是SLR(1)文法
说明：没有发现移进-规约冲突和规约-规约冲突

图 5、6、7、8 SLR(1)文法判断测试

再点击上方“LR(1)DFA 生成”按钮，即可在下方的分析结果表中查看到对应文法的 LR(0)DFA（表格形式），测试结果如下图所示：

分析结果表

	状态	状态内文法	ASSIGN	DIVIDE	EC
1	0	program-...			
2	1	assign-stmt-...	13		
3	2	statement-...			
4	3	if-stmt-...			
5	4	statement->if...			
6	5	read-stmt-...			
7	6	statement-...			
8	7	repeat-stmt-...			
9	8	statement-...			
10	9	stmt-sequenc...			
11	10	program-...			
	11	...			

图 9 LR(1)DFA 图生成测试

由于显示区域大小问题，构建的 LR(1)DFA 图（表格形式）无法显示完全，但是可以通过滑动条实现完整的查看。

点击上方的“LR(1)DFA 生成”按钮，即可在下方的分析结果表中查看到对应文法的 LR(0)DFA（表格形式），测试结果如下图所示：

分析结果表

	状态	状态内文法	ASSIGN	DIVIDE	EC
1	0	program-...			
2	1	assign-stmt-...	13		
3	2	statement-...			
4	3	if-stmt-...			
5	4	statement->if...			
6	5	read-stmt-...			
7	6	statement-...			
8	7	repeat-stmt-...			
9	8	statement-...			
10	9	stmt-sequenc...			
11	10	program-...			
	11	...			

图 10 LR(1)DFA 图生成测试（TINY 文法）

由于显示区域大小问题，构建的 LR(1)DFA 图（表格形式）无法显示完全，但是可以通过滑动条实现完整的查看。

接下来是源代码语法分析并构建语法分析表的测试，在指定输入框输入/导入测试代码的 lex 文件后，点击上方的“源代码分析”按钮，即可实现源程序语法分析表的构建，测试结果如下图所示：

分析结果表

	步骤	状态	符号栈	动作
1	0	0		...
2	1	0		...
3	2	0 5	read	ID SEMI ID ASSIGN ...
4	3	0...	read ID	SEMI ID ASSIGN NUMBER ...
5	4	0 6	read-stmt	SEMI ID ASSIGN NUMBER ...
6	5	0 6	read-stmt	SEMI ID ASSIGN NUMBER ...
7	6	0 9	statement	SEMI ID ASSIGN NUMBER ...
8	7	0 9	statement	SEMI ID ASSIGN NUMBER ...
9	8	0...	stmt-sequence	SEMI ID ASSIGN NUMBER ...
10	9	0...	stmt-sequence	SEMI ID ASSIGN NUMBER ...
11	10	0...	stmt-sequence SEMI	ID ASSIGN NUMBER SEMI ...
12	11	0...	stmt-sequence SEMI ID	ASSIGN NUMBER SEMI ID ...

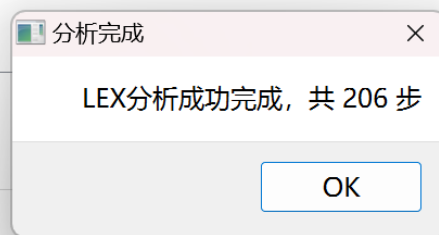


图 11 LR(1)源程序语法分析表生成 (测试 1)

分析结果表

	步骤	状态栈	序号	余数	说明
1	0	0		1...	初始
2	1	0		1...	移进
3	2	0 1	ID	...	移进: 状态1遇到'ASSIGN', 移进到状态13
4	3	0 1 13	1...	1...	移进: 状态13遇到'ID', 移进到状态14
5	4	0 1 13 14	1...	...	规约: 使用产生式 factor -> ID
6	5	0 1 13 18	1...	...	规约完成, 执行GOTO
7	6	0 1 13 18	1...	...	规约: 使用产生式 term -> factor
8	7	0 1 13 20	1...	...	规约完成, 执行GOTO
9	8	0 1 13 20	1...	...	移进: 状态20遇到'MOD', 移进到状态42
10	9	0 1 13 20 42	1...	...	规约: 使用产生式 mulop -> MOD
11	10	0 1 13 20 62	1...	...	规约完成, 执行GOTO
12	11	0 1 13 20 62	1...	...	移进: 状态62遇到'ID', 移进到状态14

分析完成

LEX分析成功完成, 共 38 步

OK

图 12 LR(1)源程序语法分析表生成（测试 2）

由于显示区域大小问题，构建的 LR(1)源程序语法分析表无法显示完全，但是可以通过滑动条实现完整的查看。

在进行源程序语法分析后，点击“可视化语法树”按钮，选择“语法树展示”，即可查看语法树如下图所示：

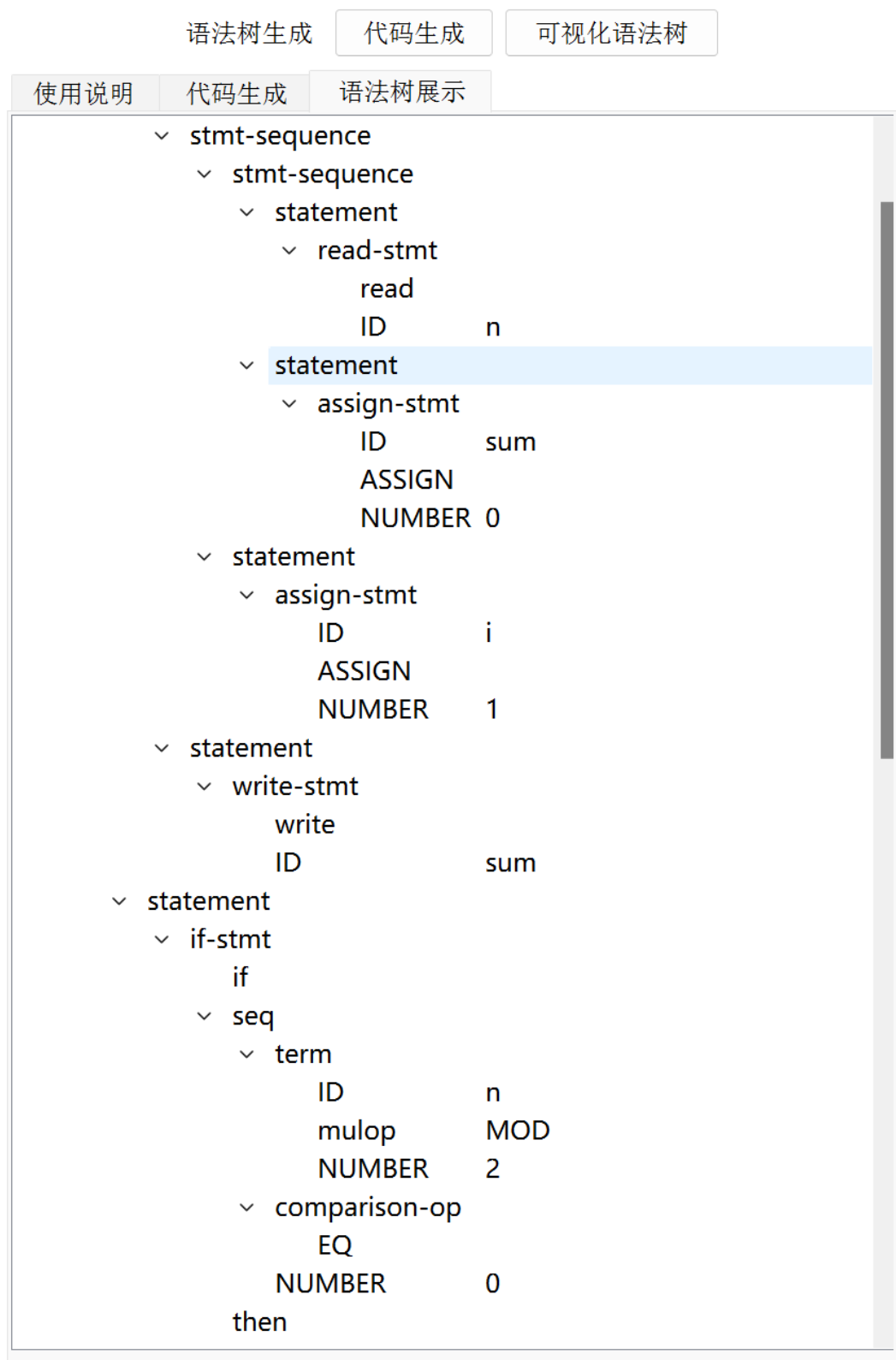


图 2-21 语法树生成测试（测试 2）



图 14 语法树生成测试（测试 2）

最后，基于测试 2 的 lex 文件内容，进行完文法分析以及语法树生成后，点击“代码生成”按钮，即可观察到中间代码生成功能的测试结果如下图所示：

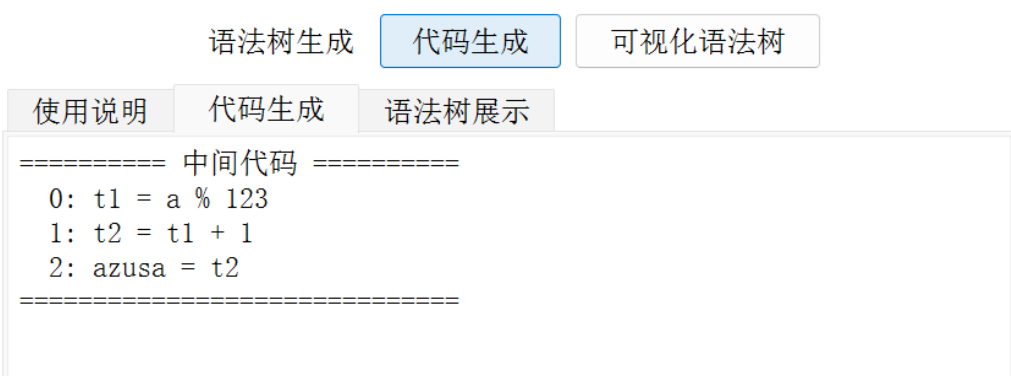


图 15 中间代码（三元组）的单元测试结果

综上，FIRST 集、FOLLOW 集、LR(0)DFA、LR(1)DFA、LR(1)分析表、LR(1)源代码语法分析表、可视化语法树均能正确生成，由于显示区域的大小问题，

这几项功能存在无法在初始情况下将结果完整展示出来的情况，但可以通过上下、左右的滚动条实现完整查看，并且已确认生成结果正确。

对于 SLR(1)语法判断分析，并未出现冲突，证明该文法是 SLR(1)文法。

测试结果：

经过测试，TINY 文法和 TINY 测试源码输入后测试所有的功能均通过，系统能实现对 TINY 文法和测试源码进行分析并构建 FIRST 集、FOLLOW 集、LR(0)DFA（表格形式展示）、LR(1)DFA（表格形式展示）、SLR(1)文法判断分析、LR(1)分析表、句子分析表、可视化语法树、中间代码，全部测试均可通过。